

AI Smart Cafeteria Waste Reduction

Kod yapısı, veri akışı, backend, web paneli, mobil uygulama, AI/ML modülleri ve kullanıcı senaryoları için düzenlenmiş teknik çalışma raporu.

Hazırlanma tarihi

12 Mayıs 2026

İncelenen ana dizin

ai_yemekhane/

İçindekiler

- [AI Smart Cafeteria Waste Reduction - Proje Çalışma Raporu](#)
 - [1. Genel Amaç](#)
 - [2. Ana Mimari](#)
 - [3. Uygulama Başlangıç Akışı](#)
 - [4. Konfigürasyon Sistemi](#)
 - [5. Veritabanı Modelleri](#)
 - [6. Route Katmanı](#)
 - [7. İş Mantığı Modülleri](#)
 - [8. Veri Akışı](#)
 - [9. Web Arayüzü](#)
 - [10. Mobil Uygulama](#)
 - [11. Scriptler](#)
 - [12. Testler](#)
 - [13. Sistemin Temel Kullanıcı Senaryoları](#)
 - [14. ML Modellerinin Dosya Yapısı](#)
 - [15. Önemli Entegrasyon Notları](#)
 - [16. Projenin Kısa Teknik Özeti](#)

AI Smart Cafeteria Waste Reduction - Proje Çalışma Raporu

Hazırlanma tarihi: 12 Mayıs 2026

İncelenen ana dizin: `ai_yemekhane/`

Bu rapor, projenin mevcut kod yapısı okunarak hazırlanmıştır. Amaç, sistemin nasıl çalıştığını, hangi modülün ne iş yaptığını, verinin sistem içinde nasıl aktığını ve kullanıcı/admin/mobil taraflarının hangi parçalarla etkileştiğini açıklamaktır.

1. Genel Amaç

Proje, üniversite yemekhanesinde yemek israfını azaltmak için geliştirilmiş bir yapay zeka destekli karar destek sistemidir. Sistem temel olarak şu işleri yapar:

- Günlük ve haftalık menüleri gösterir.
- Öğrencilerden anonim yemek puanı, yorum ve “ne kadar yemek artırdım” bildirimi toplar.
- Yönetici tarafından girilen üretim ve kalan porsiyon verilerinden gerçek israf oranını hesaplar.
- Geçmiş puanlama, üretim, maliyet ve beslenme verilerini kullanarak haftalık menü önerisi üretir.
- İsraf riski yüksek yemekleri tahmin eder.
- Uyarı, rapor, maliyet analizi, A/B test, anomali tespiti ve XAI açıklamaları üretir.
- Web arayüzü ve Expo tabanlı mobil uygulama ile kullanılabilir.

Kısaca sistem; öğrenci geri bildirimi, üretim verisi ve menü verisini birleştirip yöneticinin daha az israf çıkaracak, daha beğenilen ve daha dengeli menüler planlamasına yardımcı olur.

2. Ana Mimari

Proje iki ana uygulamadan oluşur:

- Backend ve web paneli: `ai_yemekhane/`
- Mobil uygulama: `ai_yemekhane/mobile_app/`

Backend tarafı FastAPI ile yazılmıştır. Veritabanı erişimi SQLAlchemy üzerinden yapılır. Varsayılan veritabanı SQLite’dir; `DATABASE_URL` verilirse PostgreSQL gibi başka SQLAlchemy uyumlu veritabanları da kullanılabilir.

Genel yapı:

ai_yemekhane/	
main.py	FastAPI uygulama giriş noktası
config.py	Ortam değişkenleri ve güvenlik ayarları
models.py	SQLAlchemy veritabanı modelleri
api/routes/	HTTP API ve HTML sayfa route'ları
modules/	İş mantığı, ML, rapor, analiz modülleri
templates/	Jinja2 HTML sayfaları
static/	CSS, JS, görseller
data/	Menü ve besin CSV verileri
scripts/	Veri yükleme, eğitim ve bakım scriptleri
tests/	Pytest testleri
mobile_app/	Expo React Native mobil istemci

3. Uygulama Başlangıç Akışı

Backend başlarken ana dosya `main.py` çalışır.

Çalışma sırası şöyledir:

1. `config.py` içindeki `active_config` yüklenir.
2. `.env` dosyası okunur.
3. FastAPI uygulaması oluşturulur.
4. Eski `/api/...` çağrılarını `/api/v1/...` altına yönlendiren middleware eklenir.
5. CORS ayarları yapılır.
6. `static/` klasörü web'e servis edilir.
7. `models.init_db()` çağrılır ve tablolar yoksa oluşturulur.
8. `api/routes/` altındaki bütün router'lar uygulamaya eklenir.
9. Dosya doğrudan çalıştırılırsa Uvicorn sunucusu başlar.

Varsayılan port `config.py` içinde `8001` olarak tanımlanmıştır. Komut:

```
cd ai_yemekhane
python main.py
```

Uygulama açıldığında:

- Web arayüzü: `http://localhost:8001`
- Swagger API dokümantasyonu: `http://localhost:8001/docs`

4. Konfigürasyon Sistemi

`config.py` projenin ortam ayarlarını yönetir.

Önemli değişkenler:

- `SECRET_KEY` : Admin oturum token'ı için kullanılır.
- `DEBUG` : Geliştirme modunu belirler.
- `DATABASE_URL` : Veritabanı bağlantısı. Varsayılan: `sqlite:///ai_yemekhane/yemekhane.db`
- `GEMINI_API_KEY` : Gemini chatbot ve AI geri bildirim analizi için kullanılır.
- `ADMIN_PASSWORD` : Admin panel şifresi.

- `HOST` ve `PORT` : Sunucu adresi ve portu.

Production ortamında zayıf varsayılan değerlerle başlatmayı engelleyen güvenlik kontrolü vardır. Development ortamında ise zayıf `SECRET_KEY` veya `ADMIN_PASSWORD` için sadece uyarı verir.

5. Veritabanı Modelleri

Tüm modeller `models.py` içinde tanımlıdır.

Yemek

Yemeklerin ana bilgisini tutar.

Alanları:

- `ad` : Yemek adı
- `kategori` : `corba` , `ana_yemek` , `yan_yemek` , `tatli` , `salata` , `icecek`
- `kalori` , `protein` , `karbonhidrat` , `yag` : Besin değerleri
- `birim_maliyet` : Porsiyon başı maliyet

Bu tablo menü optimizasyonu, maliyet hesaplama, besin değeri sorgusu ve simülasyonlarda kullanılır.

Menu

Günlük yemekhane menüsünü tutar.

Alanları:

- `tarih`
- `gun`
- `corba`
- `ana_yemek`
- `yan_yemek`
- `tatli`
- `salata`

Öğrenci menü ekranı, mobil menü ekranı, chatbot ve admin menü yönetimi bu tablodan veri alır.

Kullanici ve KullaniciYemekLog

Kullanıcı profili ve kullanıcı bazlı yemek tüketim kaydı için tanımlanmıştır. Mevcut ana web akışlarında puanlama anonim olduğu için sistemin kritik kısmı `MenuPuanlama` ve `UretimLog` üzerindedir.

MenuPuanlama

Öğrencilerin anonim yemek puanı ve yorumlarını tutar.

Alanları:

- `tarih`
- `yemek_adi`
- `kategori`

- `puan` : 1-5 arası yıldız
- `yorum`
- `israf_self_report` : Öğrencinin kendi israf bildirimi
 - 0: hiç
 - 1: az
 - 2: orta
 - 3: çok
- `created_at`

Bu tablo sistemin öğrenci geri bildirim kaynağıdır. Menü optimizasyonu, sentiment analizi, israf tahmini, uyarı sistemi ve chatbot bu veriden yararlanır.

UretimLog

Yönetici tarafından girilen gerçek üretim ve kalan porsiyon verisini tutar.

Alanları:

- `tarih`
- `yemek_adi`
- `kategori`
- `uretilen_porsiyon`
- `kalan_porsiyon`
- `tuketim_orani`
- `israf_orani`
- `notlar`
- `tartilan_israf_kg`
- `tartilan_israf_kaynagi`

Bu tablo gerçek israf hedefidir. İsraf ML modeli bu tablodaki `israf_orani` değerini öğrenmeye çalışır.

Alert

Otomatik uyarıları tutar.

Örnek uyarılar:

- Bir yemek 3 hafta üst üste düşük puan aldıysa kritik uyarı
- Bir kategoride genel puan düşüşü varsa bilgi uyarısı
- Ani puan düşüşü varsa dikkat uyarısı

MenuOneriLog

AI tarafından üretilen haftalık menü önerilerini saklar. A/B test modülü bu tabloyu gerçek menü ile karşılaştırır.

MenuAlternatif **ve** MenuOylama

Haftalık öğrenci oylama sistemi için kullanılır.

- `MenuAlternatif` : AI tarafından üretilen A/B/C menü alternatifleri

- `MenuOylama` : Öğrencinin anonim oy kaydı

`AnomaliKaydi`

Anomali tespit sisteminin bulduğu olağan dışı israf/tüketim olaylarını saklar.

`PushToken`

Mobil cihazların Expo push notification token'larını saklar.

6. Route Katmanı

HTTP endpoint'leri `api/routes/` altında modüler ayrılmıştır. Çoğu API `/api/v1` prefix'i ile çalışır. Eski `/api/...` çağrıları middleware tarafından otomatik olarak `/api/v1/...` haline yönlendirilir.

`pages.py`

HTML sayfalarını döndürür.

Herkese açık sayfalar:

- `/`
- `/chat`
- `/rate`
- `/today-menu`
- `/vote`

Admin şifreli sayfalar:

- `/admin`
- `/menu`
- `/report`
- `/production`
- `/production-plan`
- `/feedback-analysis`
- `/admin/simulation`

Admin girişi `ADMIN_PASSWORD` ile yapılır. Başarılı girişte `admin_token` cookie'si set edilir.

`menu.py`

Menü ve besin değerleri API'lerini sağlar.

Başlıca endpoint'ler:

- `POST /api/v1/predict-menu` : Haftalık AI menü önerisi üretir.
- `GET /api/v1/menu/ab-test` : AI önerisi ile gerçek menüyü karşılaştırır.
- `GET /api/v1/menu/optimization-weights` : Optimizasyon ağırlıklarını getirir.
- `POST /api/v1/menu/optimization-weights` : Ağırlıkları günceller.
- `GET /api/v1/nutrition/{yemek_adi}` : Besin değerlerini getirir.
- `GET /api/v1/menu/today` : Bugünün menüsünü getirir.
- `GET /api/v1/ai/menu-tracking` : AI menü etkisini takip eder.

Bu route `modules.menu_optimizer` modülünü kullanır.

`ratings.py`

Öğrenci puanlama sistemidir.

Başlıca endpoint'ler:

- `POST /api/v1/rate-meal` : Yemek puanı, yorum ve self-report israf verisi kaydeder.
- `GET /api/v1/ratings/today` : Bugünkü menü için ortalama puanları verir.
- `GET /api/v1/ratings/weekly-report` : Son 7 günlük puan raporunu verir.
- `GET /api/v1/ratings/all-rated-meals` : Son N günde puanlanmış yemekleri listeler.
- `GET /api/v1/ratings/history/{yemek_adi}` : Belirli yemeğin puan geçmişini verir.

Yeni puan kaydedildiğinde uyarı cache'i temizlenir ve WebSocket ile `new_rating` event'i yayınlanmaya çalışılır.

`production.py`

Üretim ve tüketim veri girişidir.

Başlıca endpoint'ler:

- `POST /api/v1/production-log` : Tek yemek için üretim/kalan verisi kaydeder.
- `POST /api/v1/production-log/bulk` : Toplu üretim girişi yapar.
- `GET /api/v1/production-log/today` : Bugünkü üretim loglarını getirir.
- `GET /api/v1/consumption/daily` : Günlük tüketim/israf raporu.
- `GET /api/v1/consumption/weekly` : Haftalık tüketim/israf özeti.
- `GET /api/v1/consumption/by-dish/{yemek_adi}` : Yemek bazlı tüketim geçmişi.
- `GET /api/v1/production/plan` : Geçmiş veriye göre üretim planı önerisi.
- `GET /api/v1/production/dish-recommendation/{yemek_adi}` : Tek yemek için porsiyon önerisi.

Bu route `consumption_tracker` ve `production_planner` modüllerini kullanır.

`waste.py`

İsraf analiz API'sidir.

Başlıca endpoint'ler:

- `GET /api/v1/waste/daily`
- `GET /api/v1/waste/weekly`
- `GET /api/v1/waste/by-dish/{yemek_adi}`
- `GET /api/v1/waste/model-status`
- `GET /api/v1/waste/feature-importance`
- `POST /api/v1/waste/train-model`
- `GET /api/v1/waste/production-summary`

Bu route `modules.waste_analyzer` modülünü kullanır.

`analytics.py`

Dashboard grafik ve analiz verilerini sağlar.

Başlıca endpoint'ler:

- GET /api/v1/analytics/israf-trend
- GET /api/v1/analytics/kategori-dagilim
- GET /api/v1/analytics/top-israf
- GET /api/v1/analytics/haftalik-karsilastirma
- GET /api/v1/analytics/maliyet
- GET /api/v1/analytics/maliyet-detay
- GET /api/v1/analytics/sentiment
- POST /api/v1/analytics/sentiment/train
- GET /api/v1/trends/monthly
- GET /api/v1/trends/seasonal
- GET /api/v1/trends/dish/{yemek_adi}
- GET /api/v1/alerts/active
- GET /api/v1/alerts/history
- GET /api/v1/feedback/analysis
- GET /api/v1/analytics/self-report
- GET /api/v1/analytics/donem-karsilastirma

Bu dosya birçok analiz modülünün birleştiği yerdir.

reports.py

PDF, Excel ve tahminsel analiz endpoint'lerini sağlar.

- GET /api/v1/report/pdf
- GET /api/v1/report/excel
- GET /api/v1/predictions/weekly
- GET /api/v1/predictions/dish/{yemek_adi}
- GET /api/v1/predictions/alerts

chatbot.py

Chatbot API'sidir.

- POST /api/v1/chat

Bugünkü menüyü context olarak alır, Gemini modelini lazy-load eder ve `modules.chatbot.get_response()` ile cevap üretir.

notifications.py

Mobil push notification sistemidir.

- POST /api/v1/notifications/register
- POST /api/v1/notifications/unregister
- POST /api/v1/notifications/send-daily-menu
- GET /api/v1/notifications/status
- POST /api/v1/notifications/send-waste-alert
- POST /api/v1/notifications/send-popular-meals

- `POST /api/v1/notifications/send-custom`

`tasks.py`

Uzun süren işleri arka plana almak için kullanılır.

Örnek işler:

- İsrar modelini eğitme
- Sentiment modelini eğitme
- Menü modelini eğitme
- PDF üretme

Task store bellektedir; tamamlanan/başarısız task'lar TTL ve maksimum kayıt sayısı ile temizlenir.

`websocket.py`

Admin dashboard için gerçek zamanlı event yayınlar.

Endpoint:

- `WS /api/v1/ws/dashboard`
- `GET /api/v1/ws/status`

Desteklenen event türleri:

- `new_rating`
- `waste_alert`
- `task_update`
- `menu_update`

`experiments.py`

A/B test raporlarını üretir.

- `GET /api/v1/experiments/ab-test`
- `GET /api/v1/experiments/summary`

`auto_trainer.py`

Model sağlık ve yeniden eğitim API'sidir.

- `GET /api/v1/models/health`
- `POST /api/v1/models/retrain`
- `POST /api/v1/models/retrain-async`

`model_tracker.py`

Model metrik geçmişini verir.

- `GET /api/v1/models/metrics`
- `GET /api/v1/models/metrics/latest`
- `GET /api/v1/models/metrics/degradation`

iot.py

IoT tartı entegrasyonu ve simülasyon endpoint'lerini sağlar.

- POST /api/v1/iot/weight-data
- POST /api/v1/iot/simulate
- GET /api/v1/iot/status
- GET /api/v1/iot/daily
- GET /api/v1/iot/trend

maliyet.py

Yemek maliyetleri ve israf maliyet analizidir.

- GET /api/v1/maliyet/yemekler
- PUT /api/v1/maliyet/guncelle
- PUT /api/v1/maliyet/toplu-guncelle
- GET /api/v1/maliyet/analiz
- GET /api/v1/maliyet/trend
- GET /api/v1/maliyet/detay

simulation.py

What-if simülasyon aracıdır.

- GET /api/v1/simulation/yemek-listesi
- GET /api/v1/simulation/menu-bugun
- POST /api/v1/simulation/what-if

Bir yemeği başka bir yemekle değiştirince tahmini israf, maliyet ve popülerlik farkını hesaplar.

voting.py

Haftalık menü alternatifleri ve öğrenci oylama sistemidir.

- POST /api/v1/voting/generate
- GET /api/v1/voting/alternatifler
- POST /api/v1/voting/oy-ver
- GET /api/v1/voting/sonuclar
- GET /api/v1/voting/gecmis

AI üç menü alternatifi üretir: Dengeli, Popüler, Ekonomik. Öğrenciler anonim oy verir.

xai.py

Explainable AI endpoint'leridir.

- GET /api/v1/xai/status
- GET /api/v1/xai/global
- GET /api/v1/xai/explain

SHAP ile model kararlarının nedenlerini açıklamaya çalışır.

`anomaly.py`

Anomali tespiti endpoint'leridir.

- `POST /api/v1/anomaly/detect`
- `GET /api/v1/anomaly/list`
- `GET /api/v1/anomaly/stats`
- `POST /api/v1/anomaly/{anomaly_id}/resolve`
- `GET /api/v1/anomaly/explain/{anomaly_id}`

7. İş Mantığı Modülleri

`modules/menu_optimizer.py`

Bu modül haftalık menü önerisi üretir.

Ana görevleri:

- Menü CSV verisini işler.
- Yemekleri kategoriye göre ayırır.
- Popülerlik skoru üretir veya modelden tahmin eder.
- ML modeli eğitir ve kaydeder.
- Haftalık menü optimizasyonu yapar.
- AI menü önerilerini A/B test için veritabanına kaydeder.
- AI önerisi ile gerçek menüyü karşılaştırır.

Kullandığı hedefler:

- Popülerlik yüksek olsun.
- İsraf düşük olsun.
- Beslenme dengesi korunsun.
- Maliyet hedefe yakın olsun.

Varsayılan optimizasyon ağırlıkları:

- `populerlik` : 0.35
- `israf` : 0.30
- `beslenme` : 0.20
- `maliyet` : 0.15

Menü üretirken uyguladığı kısıtlar:

- Aynı yemek haftada en fazla 2 kez seçilir.
- Her gün farklı çorba seçilmeye çalışılır.
- Aynı et türü haftada en fazla 2 kez kullanılır.
- Haftada en az 1 gün sebze ağırlıklı menü olmalıdır.
- Aynı gün alerjen grubu aşırı yoğun olmamalıdır.
- `KRITIK` uyarılı yemekler otomatik elenir.

Menü seçiminde her aday yemek için toplam skor hesaplanır:

```
toplam_skor =  
    populerlik_agirligi * populerlik_skoru  
+ israf_agirligi * (1 - israf_orani)  
+ beslenme_agirligi * beslenme_skoru  
+ maliyet_agirligi * maliyet_skoru
```

modules/waste_analyzer.py

İsraf tahmin ve raporlama modülüdür.

Bu modül iki modda çalışır:

1. Eğitilmiş model varsa ML tahmini yapar.
2. Model yoksa kural tabanlı fallback kullanır.

Kural tabanlı puan-israf dönüşümü:

- 1 puan yaklaşık yüzde 80 israf
- 2 puan yaklaşık yüzde 65 israf
- 3 puan yaklaşık yüzde 35 israf
- 4 puan yaklaşık yüzde 12 israf
- 5 puan yaklaşık yüzde 5 israf

ML modelinin hedef değişkeni:

- `UretimLog.israf_orani`

Model özellikleri arasında şunlar vardır:

- Yemek adı
- Kategori
- Haftanın günü
- Ay
- Üretilen porsiyon
- Ortalama puan
- Oy sayısı
- Önceki hafta israfı
- Popülerlik skoru
- Mevsim
- Hafta içi/hafta sonu bilgisi
- Son 3 gün israf ortalaması
- Kategori ortalama israfı
- Puan standart sapması
- Menü çeşitliliği
- Tahmini öğrenci katılımı
- Menü çekiciliği
- Gün tipi
- Önceki gün israfı
- Self-report ortalaması

Eğitimde kullanılan yaklaşım:

- RandomForestRegressor
- GradientBoostingRegressor
- XGBoost varsa XGBRegressor
- Bunlar VotingRegressor ile ensemble yapılır.
- Optuna varsa hiperparametre optimizasyonu yapılır.
- Yeterli veri varsa TimeSeriesSplit ile zaman serisi doğrulaması yapılır.

Tahmin karar sırası:

1. ML modeli
2. Gerçek üretim israf ortalaması
3. Puan tabanlı kural
4. Veri yok

`modules/production_planner.py`

Üretilecek porsiyon sayısını önerir.

Mantık:

1. Son 60 gündeki üretim ve tüketim geçmişini alır.
2. Ortalama tüketimi ve standart sapmayı hesaplar.
3. İsfraf modelinden tahmini israf oranı alır.
4. Talep tabanı oluşturur:

```
talep_tabani = ortalama_tuketim + servis_seviyesi_z * tuketim_std
```

1. ML yüksek israf bekliyorsa üretimi düşürmeye çalışan ikinci bir sinyal üretir.
2. Final öneride talep yetersizliği oluşmaması için talep tabanı korunur.
3. Öneri geçmiş ortalamadan çok sapmasın diye alt/üst sınır uygulanır.

Bu modül, mutfak yöneticisine “bu yemekten kaç porsiyon üretmeliyim” sorusuna veri destekli cevap verir.

`modules/consumption_tracker.py`

Manuel üretim ve kalan porsiyon girişlerini işler.

Yaptığı işler:

- `uretilen` ve `kalan` değerlerini doğrular.
- `kalan > uretilen` gibi hataları engeller.
- Tüketim oranını hesaplar.
- İsfraf oranını hesaplar.
- Aynı tarih ve yemek için kayıt varsa günceller.
- Günlük, haftalık ve yemek bazlı tüketim raporları üretir.

`modules/chatbot.py`

Gemini tabanlı yemekhane asistanıdır.

Görevleri:

- Bugünkü menüyü cevaplamak
- Yemeklerin besin değerlerini anlatmak
- Yemek puanlarını göstermek
- Kullanıcı adına puanlama yapmak
- İsraf, uyarı ve maliyet bilgisi vermek
- Haftalık menü ve menü karşılaştırması yapmak
- Geri bildirim analizi, popülerlik ve maliyet analizi araçlarını kullanmak

Gemini function calling kullanır. Tanımlı araçlar arasında şunlar vardır:

- `get_nutrition`
- `get_today_menu`
- `get_meal_ratings`
- `rate_meal`
- `get_waste_info`
- `get_alerts_info`
- `get_weekly_menu`
- `get_menu_comparison`
- `analyze_student_feedback`
- `get_popularity_ranking`
- `get_cost_analysis`

Konuşma geçmişi bellekte LRU mantığıyla tutulur. Gemini API anahtarı yoksa placeholder/fallback cevaplar devreye girer.

`modules/sentiment_analyzer.py`

Türkçe yorum duygu analizi yapar.

Yaklaşım:

- Kelime tabanlı pozitif/negatif skorlar
- Emoji etkisi
- N-gram ve olumsuzluk kontrolü
- Aspect-based sentiment
- Yeterli veri varsa TF-IDF + LogisticRegression eğitimi

Çıktılar:

- Pozitif/negatif/nötr dağılım
- Yemek bazlı duygu
- Tema ve şikayet başlıkları
- Eğitilmiş model varsa model tabanlı tahmin

`modules/feedback_analyzer.py`

Öğrenci geri bildirimlerini yönetim raporuna dönüştürür.

Veri kaynakları:

- `MenuPuanlama` : puan ve yorum
- `UretimLog` : tüketim ve israf

Çalışma şekli:

1. Son N günün puanlarını, yorumlarını ve üretim verilerini toplar.
2. Gemini için yapılandırılmış prompt üretir.
3. Gemini'den JSON formatında analiz bekler.
4. Gemini yoksa puan tabanlı yerel fallback analiz yapar.

Ürettiği sonuçlar:

- Duygu dağılımı
- En sık temalar
- Yemek duygu skorları
- Menü aksiyon önerileri: kaldır, azalt, koru, artır
- Genel değerlendirme

`modules/alert_system.py`

Otomatik uyarı sistemidir.

Kurallar:

- Son 3 hafta ortalama puan 2.5 altındaysa `KRITIK`
- Son 2 hafta ortalama puan 3.0 altındaysa `UYARI`
- Puan yüzde 30 veya daha fazla aniden düşerse `DIKKAT`
- Kategori genelinde yüzde 15 veya daha fazla düşüş varsa `BILGI`

Aktif uyarılar 5 dakika TTL cache ile saklanır. Yeni puanlama geldiğinde cache temizlenir.

`modules/predictive_analyzer.py`

Gelecek hafta israf riskini tahmin eder.

Yöntem:

- Son haftaların ortalama puanlarından israf skoruna dönüşüm yapılır.
- Ağırlıklı hareketli ortalama kullanılır.
- Trend yönü hesaplanır.
- Düşük puanlı riskli yemekler belirlenir.

Çıktılar:

- Gelecek hafta tahmini israf skoru
- Trend yönü
- Güven seviyesi
- Riskli yemekler
- Yönetim önerileri

`modules/trend_analyzer.py`

Uzun dönem trend analizi yapar.

Ürettiği analizler:

- Son N gün genel puan trendi
- Yemek bazlı ortalama puanlar
- Mevsimsel popülerlik
- Belirli yemeğin uzun vadeli trendi
- Basit z-score mantığıyla puan anomalileri

`modules/pdf_report.py`

PDF ve Excel rapor üretir.

Kullanılan araçlar:

- ReportLab
- Matplotlib
- OpenPyXL

Rapor içeriği:

- Haftalık özet
- Aylık özet
- Puanlama tabloları
- Günlük trend grafikleri
- Kategori analizi
- İsraf trendi
- Maliyet bölümü
- Dönem karşılaştırmaları

Windows ortamında Türkçe karakter desteği için Arial fontunu kaydetmeye çalışır.

`modules/notification_service.py`

Expo Push Notification entegrasyonudur.

Yaptığı işler:

- Expo token kaydeder.
- Token pasifleştirir.
- Aktif token listesini alır.
- Günlük menü bildirimi gönderir.
- İsraf uyarısı bildirimi gönderir.
- Haftanın popüler yemeklerini bildirir.
- Admin özel bildirimi gönderir.

Token'lar `PushToken` tablosunda kalıcı saklanır.

`modules/experiment_engine.py`

A/B test motorudur.

Karşılaştırılan gruplar:

- A grubu: Gerçek menü (Menu)
- B grubu: AI önerisi (MenuÖneriLog)

Metrikler:

- Ortalama israf
- Ortalama puan
- Örneklem sayısı

Scipy varsa Welch t-test uygulanır. Scipy yoksa basit fark karşılaştırması yapılır.

`modules/xai_explainer.py`

Explainable AI modülüdür.

Görevleri:

- Modelin genel feature importance listesini verir.
- SHAP varsa global SHAP açıklaması üretir.
- Tek yemek için “bu tahmini hangi faktörler etkiledi” açıklaması üretir.

Model yoksa veya SHAP kurulu değilse hata/fallback mesajı döner.

`modules/anomaly_detector.py`

Anomali tespit modülüdür.

Kullandığı yöntemler:

- Z-score
- Isolation Forest
- Kategori sapması analizi

Tespit ettiği olaylar:

- Beklenenden yüksek israf
- Beklenenden düşük tüketim
- Kategori ortalamasından sapma
- Olağan dışı düşüş/yükseliş

Tespit edilen anomaliler `AnomaliKaydi` tablosuna yazılır. Yönetici bunları çözülmüş olarak işaretleyebilir.

`modules/auto_trainer.py`

Otomatik model yeniden eğitim sistemidir.

Desteklenen modeller:

- waste
- sentiment

Mantık:

1. Son eğitimden bu yana yeni veri sayısını kontrol eder.

2. Eşik aşılmışsa modeli yeniden eğitir.
3. Eski modeli yedekler.
4. Yeni eğitim başarısız olursa eski modeli geri yükler.

Bu yaklaşım champion-challenger yaklaşımıdır.

`modules/model_tracker.py`

Model eğitim metriklerini JSON dosyasında saklar.

Saklanan metrik örnekleri:

- MAE
- RMSE
- R2
- Accuracy
- F1

Son iki eğitim arasında performans düşüşü olup olmadığını kontrol eder.

`modules/iot_simulator.py`

Gerçek IoT tartı cihazı olmadan tartı verisi üretir.

Varsayımlar:

- Ortalama porsiyon ağırlığı: 0.35 kg
- Rastgele sapma: yüzde 15
- Minimum israf: 0.1 kg

Formül:

```
uretilen_kg = uretilen_porsiyon * 0.35
baz_israf_kg = uretilen_kg * israf_orani
simule_kg = baz_israf_kg * rastgele_sapma
```

Sonuç `UretimLog.tartilan_israf_kg` alanına yazılır ve kaynak `simulasyon` olur.

8. Veri Akışı

8.1 Menü Verisi Akışı

1. `data/menu_data.csv` ve `data/nutrition_data.csv` dosyaları hazırlanır.
2. `scripts/data/load_menu_data.py` ve `scripts/data/seed_data.py` ile veritabanına aktarılır.
3. `Menu` tablosu günlük menüleri tutar.
4. `Yemek` tablosu yemeklerin besin ve maliyet bilgisini tutar.
5. Web ve mobil uygulama `/api/v1/menu/today` ile bugünkü menüyü gösterir.

8.2 Puanlama Akışı

1. Öğrenci web veya mobil uygulamada yemeği seçer.
2. 1-5 arası puan verir.

3. İsteğe bağlı yorum yazar.
4. Ne kadar yemek artırdığını self-report olarak seçer.
5. `POST /api/v1/rate-meal` çağrılır.
6. Veri `MenuPuanlama` tablosuna yazılır.
7. Alert cache temizlenir.
8. Dashboard bağlıysa WebSocket ile yeni puanlama event'i gider.

8.3 Üretim ve Gerçek İsraf Akışı

1. Yönetici admin panelinden üretilen ve kalan porsiyonları girer.
2. `production.py` route'u `consumption_tracker.save_production_log()` fonksiyonunu çağırır.
3. Sistem tüketim oranını hesaplar:

```
tuketim_orani = (uretilen - kalan) / uretilen * 100
```

1. Sistem israf oranını hesaplar:

```
israf_orani = kalan / uretilen * 100
```

1. Sonuç `UretimLog` tablosuna yazılır.
2. İsraf analizi, üretim planlama, maliyet analizi ve model eğitimi bu veriyi kullanır.

8.4 İsraf Tahmin Akışı

1. Sistem yemek adı, kategori, tarih, puan, üretim verisi ve geçmiş sinyalleri toplar.
2. Eğitilmiş `waste_predictor.joblib` varsa model tahmini yapılır.
3. Model yoksa gerçek üretim ortalaması kullanılır.
4. O da yoksa puan tabanlı kural kullanılır.
5. Hiç veri yoksa sonuç `veri_yok` olur.

8.5 AI Menü Önerisi Akışı

1. Yönetici haftalık menü önerisi ister.
2. `POST /api/v1/predict-menu` çağrılır.
3. Sistem tüm yemekleri `Yemek` tablosundan alır.
4. Menü popülerlik modeli diskten yüklenir.
5. `generate_weekly_menu()` her gün ve kategori için adayları skorlar.
6. Kısıtlar uygulanır.
7. En yüksek skorlu yemekler haftalık menüye yerleştirilir.
8. Menü skoru hesaplanır.
9. Başlangıç tarihi verilmişse öneri `MenuOneriLog` tablosuna kaydedilir.

8.6 Raporlama Akışı

1. Yönetici rapor ister.
2. PDF için `generate_weekly_pdf()` veya `generate_monthly_pdf()` çalışır.
3. Excel için `generate_excel_report()` çalışır.

4. Veriler `MenuPuanlama` , `UretimLog` ve `Yemek` tablolarından toplanır.
5. PDF/Excel dosyası `StreamingResponse` ile indirilir.

9. Web Arayüzü

HTML sayfaları `templates/` altındadır.

Öğrenci tarafı:

- `index.html` : Ana sayfa
- `today_menu.html` : Günün menüsü
- `rate.html` : Puanlama
- `chat.html` : Chatbot
- `vote.html` : Menü oylama

Admin tarafı:

- `admin_login.html` : Giriş
- `admin.html` : Dashboard
- `menu.html` : Menü yönetimi
- `report.html` : Raporlama
- `production_entry.html` : Üretim veri girişi
- `production_plan.html` : Üretim planı
- `feedback_analysis.html` : Geri bildirim analizi
- `simulation.html` : What-if simülasyon

`static/js/` ve `static/css/` klasörleri arayüz davranışlarını ve görünümünü sağlar. Sayfalar API endpoint'lerini çağırarak verileri dinamik alır.

10. Mobil Uygulama

Mobil uygulama Expo/React Native ile yazılmıştır.

Ana dosya:

- `mobile_app/App.js`

API helper:

- `mobile_app/src/api.js`

Mobil API helper eski `/api/...` path'lerini otomatik `/api/v1/...` haline çevirir.

Ana sekmeler:

- `MenuScreen`
- `RateScreen`
- `StatsScreen`
- `ChatScreen`

MenuScreen

Bugünkü menüyü getirir.

Kullandığı endpoint:

- `/api/menu/today`

Offline durumda son menüyü cache'ten gösterebilir.

RateScreen

Bugünkü menüyü alır, kullanıcıya menüden yemek seçtirir ve puanlama gönderir.

Kullandığı endpoint'ler:

- `/api/menu/today`
- `/api/rate-meal`

Self-report israf bildirimi de gönderir.

ChatScreen

Chatbot mesajlaşma ekranıdır.

Kullandığı endpoint:

- `/api/chat`

StatsScreen

İstatistik panelidir.

Kullandığı endpoint'ler:

- `/api/analytics/maliyet?gun=30`
- `/api/analytics/maliyet-detay?gun=30`
- `/api/analytics/self-report?gun=30`
- `/api/v1/iot/status`
- `/api/v1/maliyet/analiz`

Dikkat: Kodda ayrıca `/api/analytics/haftalik-israf` çağrısı var. Backend tarafında görünen karşılık endpoint `/api/v1/analytics/haftalik-karsilastirma`. Bu isim farkı mobil istatistik ekranında haftalık karşılaştırma verisinin gelmemesine neden olabilir.

Push Notification

`App.js` içinde Expo notification kurulumu vardır.

Akış:

1. Kullanıcıdan bildirim izni istenir.
2. Android için notification channel oluşturulur.
3. Expo push token alınır.
4. Token backend'e `/api/notifications/register` ile gönderilir.
5. Günlük menü bildirimi veya özel bildirimler backend üzerinden gönderilebilir.

11. Scriptler

`scripts/data/` veri hazırlama ve bakım scriptlerini içerir.

Örnekler:

- `load_menu_data.py` : Menü verisini veritabanına yükler.
- `seed_data.py` : Örnek verileri oluşturur.
- `generate_production_logs.py` : Üretim logu üretir.
- `fix_all_data.py` , `fix_menu_csv.py` , `fix_ratings.py` : Veri düzeltme scriptleri.
- `import_xlsx_menu.py` , `convert_xlsx_to_csv.py` : Excel/CSV dönüşümleri.

`scripts/training/` model eğitim scriptlerini içerir.

Örnekler:

- `train_menu_model.py`
- `train_waste_model.py`

12. Testler

Testler `tests/` altındadır.

Kapsanan başlıklar:

- API endpoint'leri
- Chatbot
- İsraf analizleri
- Menü optimizasyonu
- Maliyet
- IoT simülatörü
- Anomali tespiti
- Auto trainer
- Experiment engine
- Sentiment
- Voting
- XAI

Çalıştırma:

```
cd ai_yemekhane
pytest tests/ -v
```

13. Sistemin Temel Kullanıcı Senaryoları

Senaryo 1: Öğrenci bugünkü menüyü görür

1. Öğrenci `/today-menu` sayfasını veya mobil `Menu` sekmesini açar.

2. Frontend `/api/v1/menu/today` çağırır.
3. Backend `Menu` tablosunda bugünün tarihini arar.
4. Menü varsa JSON olarak döner.
5. Arayüz menüyü kategorilere göre gösterir.

Senaryo 2: Öğrenci yemek puanlar

1. Öğrenci `rate` ekranında bugünkü menüden yemek seçer.
2. Puan, yorum ve self-report israf bilgisini gönderir.
3. Backend veriyi `MenuPuanlama` tablosuna yazar.
4. Uyarı sistemi cache'i temizlenir.
5. Dashboard canlı bağlıysa WebSocket event'i gider.

Senaryo 3: Yönetici üretim verisi girer

1. Yönetici `/production` sayfasına girer.
2. Her yemek için üretilen ve kalan porsiyon sayısını girer.
3. Sistem tüketim ve israf oranını otomatik hesaplar.
4. `UretimLog` tablosu güncellenir.
5. Analiz, rapor, maliyet ve model eğitim modülleri bu veriyi kullanır.

Senaryo 4: Yönetici AI haftalık menü önerisi ister

1. Yönetici menü önerisi endpoint'ini tetikler.
2. Sistem yemek listesini, puan sinyallerini, üretim/israf sinyallerini ve maliyetleri toplar.
3. Menü optimizier her gün için adayları skorlar.
4. Kısıtlar uygulanır.
5. 5 günlük öneri döner.
6. Başlangıç tarihi verilmişse öneri A/B test için kaydedilir.

Senaryo 5: Chatbot menü veya besin sorusunu yanıtlar

1. Kullanıcı chat ekranına mesaj yazar.
2. Backend bugünkü menüyü context olarak hazırlar.
3. Gemini modeline sistem prompt'u ve araç tanımları verilir.
4. Gemini gerekirse araç çağırır.
5. Araç sonucu ile Türkçe cevap döner.

Senaryo 6: Rapor oluşturulur

1. Yönetici rapor sayfasından PDF veya Excel ister.
2. Backend ilgili rapor fonksiyonunu çağırır.
3. Veriler veritabanından alınır.
4. PDF/Excel bellekte oluşturulur.
5. Dosya response olarak indirilir.

14. ML Modellerinin Dosya Yapısı

Model dosyaları `models_data/` altında beklenir veya oluşturulur.

Örnek dosyalar:

- `waste_predictor.joblib`
- `waste_metrics.json`
- `waste_feature_importance.json`
- `menu_popularity_model.joblib`
- `label_encoders.joblib`
- `feature_columns.joblib`
- `optimization_weights.json`
- `sentiment_tfidf_model.joblib`
- `model_metrics.json`
- `auto_trainer_state.json`

Bu dosyalar model eğitimi ve model sağlık takibi için kullanılır.

15. Önemli Entegrasyon Notları

- Backend varsayılan portu `8001` olarak ayarlanmış. Mobil README örneklerinde `8000` ve Android emulator için `10.0.2.2` geçiyor. Mobil testte `EXPO_PUBLIC_API_BASE_URL` backend'in gerçek portuna göre ayarlanmalı.
- `main.py` eski `/api/...` endpoint'lerini `/api/v1/...` endpoint'lerine yönlendiriyor. Mobil uygulama da ayrıca path'i `/api/v1` haline getiriyor.
- Mobil `StatsScreen` içinde `/api/analytics/haftalik-israf` çağrısı var; backend route listesinde bunun yerine `/api/v1/analytics/haftalik-karsilastirma` mevcut.
- Gemini kullanan bölümler için `GEMINI_API_KEY` gerekir. Anahtar yoksa bazı modüller fallback cevap/analiz üretir.
- SHAP açıklamaları için `shap` paketinin kurulmuş ve israf modelinin eğitilmiş olması gerekir.
- XGBoost ve Optuna opsiyonel olarak kullanılır; kuruluysa israf modeli daha gelişmiş eğitim yapar.

16. Projenin Kısa Teknik Özeti

Bu proje, FastAPI tabanlı bir backend üzerinde çalışan, veritabanı destekli ve modüler bir yemekhane israf azaltma sistemidir. Öğrenciden gelen puan, yorum ve israf self-report verileri `MenuPuanlama` tablosuna; yöneticiden gelen gerçek üretim/kalan verileri `UretimLog` tablosuna yazılır. Bu iki ana veri kaynağı; menü optimizasyonu, israf tahmini, maliyet analizi, duygu analizi, üretim planlama, raporlama, anomali tespiti ve chatbot modülleri tarafından kullanılır.

Sistemin en kritik noktası `UretimLog.israf_orani` alanıdır; gerçek israf hedefi buradan gelir. `MenuPuanlama` ise öğrenci memnuniyetini, yorumları ve self-report israf sinyali verir. AI menü önerisi bu sinyalleri popülerlik, beslenme, maliyet ve israf hedefleriyle birleştirir. Yönetici paneli karar destek aracı gibi çalışır; mobil uygulama ise öğrenci tarafında menü görüntüleme, puanlama, chat ve istatistik deneyimi sunar.